HNSW with Accuracy Guarantees Using Graph Spanners

Anonymous Author(s)

ABSTRACT

Hierarchical Navigable Small World (HNSW) graphs serve as the industry standard due to their logarithmic complexity and strong empirical performance. However, HNSW relies on greedy graph traversal, a heuristic that provides no theoretical guarantees of correctness. In this paper, we propose a novel "Certify-then-Rectify" framework that bridges the gap between the speed of heuristic search and the rigor of exact retrieval. Rather than discarding HNSW, our approach first employs a distribution-free statistical certifier to dynamically evaluate the quality of a standard HNSW search with minimal overhead. If the certification indicates that the retrieved neighbors are of low quality, the framework safely escalates to a rigorous exact recovery algorithm. To make this exact recovery computationally feasible, we reinterpret the HNSW graph as a geometric spanner and utilize Extreme Value Theory to stochastically estimate its maximum empirical stretch factor. This allows us to mathematically bound the maximum distance of true nearest neighbors. Furthermore, we successfully extend these theoretical guarantees to filtered search scenarios. Extensive evaluations on benchmark datasets demonstrate that our tiered framework delivers the average-case speed of HNSW while ensuring the worst-case correctness of exact search.

ACM Reference Format:
Anonymous Author(s). 2026. HNSW with Accuracy Guarantees Using Graph Spanners. In *Proceedings of ACM Conference (Conference'17).* ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Nearest Neighbor Search (NNS) is the computational engine behind modern AI, driving workloads ranging from Large Language Model (LLM) retrieval to molecular simulations. As datasets scale into the billions of vectors, exact linear scanning becomes computationally prohibitive, necessitating Approximate Nearest Neighbor (ANN) algorithms. Among these, Hierarchical Navigable Small World (HNSW) graphs [39, 42] have become the industry standard due to their logarithmic complexity and generally strong empirical performance.

However, HNSW relies on a greedy traversal of a proximity graph—a heuristic that offers no theoretical guarantee of recall [2, 13, 47, 57]. While HNSW performs well empirically, it is highly sensitive to hyperparameter selection (e.g., beam width, construction depth). In high-dimensional spaces, greedy search frequently gets trapped in local minima, failing to retrieve the true nearest neighbor due to graph topology disconnects or the "curse of dimensionality." While a 95% recall rate is acceptable for e-commerce, it is fundamentally inadequate for logic-driven and scientific domains where a single missing entry can invalidate a result.

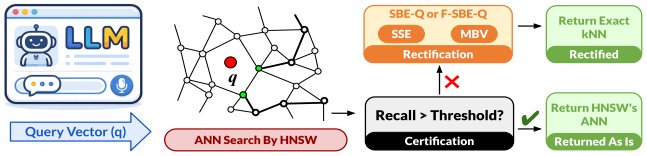

Figure 1: Probabilistic Exact Search Framework

The demand for **Exact k-NN** is growing rapidly in complex, high-stakes applications: In training perception models, identifying "hard negatives"—images that look safe but are actually dangerous—is critical [15, 27]. If an ANN algorithm fails to retrieve these specific edge cases during data mining, the training set remains noisy, compromising safety. Exact retrieval is required to rigorously clean training distributions. In systems combining neural retrieval with symbolic logic or LLMs, vector search acts as a predicate selector. Missing a specific "bridge" entity (e.g., a specific legal precedent or protein structure) leads to "hallucinations" or false logical implications because the reasoning engine operates on incomplete premises [36, 50].

Existing attempts to mitigate these failures by simply increasing or predicting [13, 21] HNSW hyperparameters yield diminishing returns, incurring latency penalties without ever providing a guarantee of correctness. Moreover, they impose retraining overheads under distribution shifts.

We propose a novel framework that bridges the gap between the speed of heuristic search and the rigor of exact retrieval. We acknowledge that while our proposed exact search method, **MBV** (Metric Bound Verification) (utilizing our proposed Stretch Bounded Expansion from Query), accurately provides a formal probabilistic guarantee, it is computationally slower than heuristic HNSW. Therefore, rather than discarding HNSW, we wrap it in a **Certify-then-Rectify (CTR)** framework.

Our approach allows users to first **certify** the quality of a standard HNSW search result with negligible overhead. If—and only if—the certification indicates that the retrieved neighbors are of low quality, we trigger the computationally heavier MBV algorithm to identify the exact k-NN. This tiered strategy provides the best of both worlds: the average-case speed of HNSW and the worst-case correctness of exact search. Our contributions are structured as follows (see Figure 1):

1. Stochastic Stretch Estimation (SSE):. To enable exact search, we reinterpret the HNSW graph as a geometric spanner. We introduce SSE, a sampling algorithm that accurately approximates the graph's stretch factor t. This factor allows us to mathematically bound the maximum distance the true nearest neighbor could reside from our approximate candidates, establishing a dynamic search radius.

Conference'17, July 2017, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM…$15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

2. Stretch-Bounded Expansion (SBE-Q):. We present SBE-Q a technique to recover the exact neighbors. Unlike greedy heuristic descent, SBE-Q performs a principled expansion.

- **SBE from the Query Point (SBE-Q):** We confine the search to a radius defined by the distance to the k-th approximate neighbor (d_k) multiplied by the stretch factor t (i.e., radius=$t \times d_k$). This guarantees the discovery of the true nearest neighbors.
- **Reservoir Sampling:** To ensure SBE-Q remains theoretically sound, we use reservoir sampling to virtually "insert" the query point q into the stretch estimation sample without altering the graph's global stretch properties.

We also explore a baseline variant, SBE-NN (Expansion from Nearest Neighbor), but find SBE-Q significantly more efficient due to its tighter bounding radius.

3. Metric Bound Verification (MBV):. Finally, we refine the candidate set obtained by SBE-Q using triangular inequality bounds derived from the stretch factor. This step recycles distance computations from the initial search to strictly filter points, ensuring the final output is the exact k-NN set. Our approach will work for any distance metric that directly or indirectly satisfies triangle inequality.

4. Utility in Filtered Search: We further demonstrate that SBE-Q naturally adapts to filtered search (e.g., "nearest vector where property P is true"), addressing scenarios where filtering fractures the small-world topology. Our techniques can be utilized within extensions like ACORN [42] to prevent traversals from getting stranded in disconnected subgraphs.

5. Certification of HNSW Results: We introduce a post-hoc, distribution-free statistical certification framework for vanilla HNSW. By leveraging Conformal Risk Control (CRC) [3, 4] and Learn-then-Test (LTT) [5] procedures, we construct a rigorous certifier that evaluates a distance-margin score derived purely from HNSW's internal search state. This approach establishes finite-sample statistical guarantees on expected recall shortfall and provides per-query probabilistic bounds. Ultimately, this certifier acts as a dynamic gatekeeper with minimal computational overhead, determining whether to confidently accept the fast heuristic results or to safely escalate to exact recovery via MBV. The entire approach is realized as a lightweight wrapper around existing HNSW implementations, facilitating ease of use.

2 PRELIMINARIES

In this section, we formalize the problem of Nearest Neighbor Search (NNS) and the structure of Hierarchical Navigable Small World (HNSW) graphs. We establish the notation for the search procedure and its computational artifacts, and we introduce the concept of graph spanners and stretch factors, which form the theoretical basis of our proposed method.

Let $\mathcal{X}$ be a dataset of n points in a d-dimensional vector space equipped with a distance metric $dist(\cdot,\cdot)$ (typically Euclidean distance). Given a query q and an integer k, the Exact k-Nearest Neighbor (k-NN) problem is to find the set $\mathcal{N}_k^*(q) \subseteq \mathcal{X}$ such that $|\mathcal{N}_k^*(q)| = k$ and $\forall x \in \mathcal{N}_k^*(q), \forall y \in \mathcal{X} \setminus \mathcal{N}_k^*(q), dist(q,x) \leq dist(q,y)$.

The HNSW index is a hierarchical graph structure approximating the proximity of points in $\mathcal{X}$. It consists of a series of layers $L_0, L_1, \ldots, L_{l_{max}}$, where layer L_0 contains all data points, and each subsequent layer L_i contains a subset of the points from layer L_{i-1}, forming a hierarchy. Formally, an HNSW structure is a tuple of graphs $\mathcal{H} = (G_0, G_1, \ldots, G_{l_{max}})$. Each graph $G_i = (V_i, E_i)$ is defined such that $V_i \subseteq \mathcal{X}$. The edges E_i connect points within the same layer based on proximity heuristics. The hierarchy allows for logarithmic search complexity by initiating the search at the top layer (coarse granularity) and progressively refining the search region as traversal moves down to layer L_0 (fine granularity).

Construction proceeds iteratively by inserting points $x \in \mathcal{X}$. For each x, a maximum layer $l(x) \leq l_{max}$ is selected randomly based on an exponential distribution. The point is inserted into all graphs $G_0, G_1, \ldots, G_{l(x)}$. Connections in each layer are established using a heuristic selection strategy that balances proximity with spatial diversity (connectivity) to ensure the "small world" property, typically bounded by a maximum degree parameter M.

The search for the k-NN of a query q in HNSW is a greedy traversal. Starting from an entry point e_{top} in the top layer $L_{l_{max}}$, the algorithm greedily moves to the neighbor minimizing $dist(q,\cdot)$ until a local minimum is reached. This process repeats down to layer L_0. In layer L_0, a beam search is conducted with a beam width parameter ef. Let this process be denoted as Search(q,k,ef). The output of this heuristic search is an approximate set $\mathcal{N}_k(q) \subseteq \mathcal{X}$, where often $|\mathcal{N}_k(q)| < k$ and $\mathcal{N}_k(q) \cap \mathcal{N}_k^*(q)$ may be empty in worst-case scenarios. Crucially, our approach utilizes the artifacts generated during this traversal. We define the Search Trace $\mathcal{T}(q)$ as the set of all points for which the distance to q was computed during the search:

$$\mathcal{T}(q) = \{x \in \mathcal{X} \mid dist(q,x) \text{ was computed during Search}(q,k,ef)\}.$$

This set includes the visited nodes and their immediate neighbors evaluated during the beam search. To bound the error of the approximate search, we analyze the bottom-layer graph G_0 as a geometric spanner.

Definition 2.1. A weighted graph $G = (V,E)$, where vertices are points in a metric space $(\mathcal{X}, dist)$, is a t-spanner if, for every pair of vertices $u, v \in V$, the shortest path distance in the graph, denoted $d_G(u,v)$, satisfies:

$$dist(u,v) \leq d_G(u,v) \leq t \cdot dist(u,v).$$

The value $t \geq 1$ is called the stretch factor (or dilation) of the spanner.

In the context of NNS, the stretch factor limits how much the graph topology distorts the true metric space. If G_0 were a perfect 1-spanner (a complete graph with edge weights equal to metric distances), greedy search would be exact. The sparsification required for efficiency introduces distortion. While HNSW construction heuristics do not theoretically guarantee a constant stretch factor t for arbitrary worst-case inputs (i.e., it is not a t-spanner by design in the traditional computational geometry sense), any realized finite HNSW graph instance G_0 possesses an intrinsic, maximum empirical stretch t_{emp} defined as:

$$t_{emp} = \max_{u,v \in \mathcal{X}} \frac{d_{G_0}(u,v)}{dist(u,v)}.$$

This property is pivotal: if we know (or estimate) t_{emp}, we can relate the graph distance—which we can explore efficiently via Breadth-First Search (BFS) or Dijkstra—to the unknown metric distance $dist(q, x)$, thereby establishing a stopping condition for search expansion that guarantees the inclusion of $N_k^*(q)$.

3 STOCHASTIC STRETCH ESTIMATION

For graphs of realistic sizes, computing the stretch factor exactly is not computationally feasible, as it would involve computations quadratic to the number of nodes in the graph. We, in turn, present sampling based algorithms to estimate the stretch factor of an HNSW graph G.

Let S be a random variable representing the stretch factor of a uniformly random pair (u, v) from $V \times V \setminus \{(v, v) : v \in V\}$. Rather than estimating arbitrary percentiles, our goal is to estimate the absolute maximum empirical stretch factor via a finite sample.

Definition 3.1 (Sample Block Maxima). Let the total sampled pairs be divided into m blocks of size n. Let $M_{n,i} = \max\{S_{i,1}, \ldots, S_{i,n}\}$ be the maximum stretch factor observed in the i-th block.

Theorem 3.2 (EVT Maximum Stretch Bound). *According to the Fisher-Tippett-Gnedenko theorem [25], the distribution of the maximum of a large sequence of independent, identically distributed random variables converges to a specific family of distributions. For the right tail of the stretch factor distribution, the cumulative distribution function (CDF) of the block maxima converges to the Generalized Extreme Value (GEV) distribution:*

$$H(s; \mu, \sigma, \xi) = \exp\left(-\left[1 + \xi\left(\frac{s-\mu}{\sigma}\right)\right]^{-1/\xi}\right)$$

where μ is the location parameter, $\sigma > 0$ is the scale parameter, and ξ is the shape parameter.

By modeling the "right tail" (the largest observed stretch factors) of our sample, we can accurately estimate the unobserved probability density of extreme stretch distortions. For a given confidence level $\beta \in (0, 1)$, the estimated maximum stretch factor t is defined as the return level derived from the inverse CDF of the fitted GEV distribution:

$$t = \mu - \frac{\sigma}{\xi}\left[1 - (-\ln \beta)^{-\xi}\right]$$

Thus, with confidence β, the true empirical stretch factor of the graph is bounded by t.

3.1 Practical Application of the EVT Bound

In practice, applying Theorem 3.2 requires a systematic sampling and fitting procedure prior to initiating the search. First, we generate a large pool of independent stretch factor samples by computing the exact metric distance and the shortest-path graph distance for random pairs of vertices in G. This pool is then partitioned into m distinct blocks of size n. We extract the maximum stretch value from each block, resulting in a specialized dataset composed entirely of block maxima. Next, we apply Maximum Likelihood Estimation (MLE) to this dataset to find the optimal parameters for the Generalized Extreme Value distribution: the location μ, scale σ, and shape ξ. Once the MLE converges and the parameters are established, we substitute them—along with our target confidence level β (e.g., 0.99)—into the GEV return level equation. The resulting value, t, acts as the rigorously defined operating stretch factor for our subsequent search expansions, ensuring the search radius dynamically adapts to the graph's most extreme topological distortions.

4 GRAPH BASED PRUNING TECHNIQUES

4.1 Stretch Bounded Expansion from Nearest Neighbor (SBE-NN)

We consider $(\mathcal{X}, dist)$ to be a metric space, where $G_0 = (\mathcal{X}, E)$ corresponds to the bottom-layer HNSW graph. Notably, this can be viewed as a *weighted* graph where each edge $(u, v) \in E$ has weight $dist(u, v)$. We denote the shortest-path distance in G_0 (sum of edge weights) as $d_{G_0}(u, v)$.

Assuming G_0 to be a t-spanner (globally or locally in the vicinity of q) in the sense that for all relevant pairs u, v:

$$dist(u, v) \le d_{G_0}(u, v) \le t \cdot dist(u, v)$$

To analyze the search expansion, we must distinguish between the approximate candidates returned by the HNSW index and the unknown ground truth.

For a given query q, let the ordered set of k candidates returned by the initial HNSW search be $\{x_{(1)}, x_{(2)}, \ldots, x_{(k)}\}$. We define the distances to the nearest and k-th candidates as:

$$\hat{d}_1 = dist(q, x_{(1)}) \quad \text{and} \quad \hat{d}_k = dist(q, x_{(k)})$$

Let $N_k^*(q)$ denote the set of the true top-k nearest neighbors. Let d_k^* denote the distance to the true k-th nearest neighbor:

$$d_k^* = \max_{x \in N_k^*(q)} dist(q, x)$$

Since the HNSW candidates $\{x_{(1)}, \ldots, x_{(k)}\}$ form a valid subset of $\mathcal{X}$ of size k, and the true set $N_k^*(q)$ minimizes the sum of distances, it necessarily holds that the true k-th distance is bounded by the approximate k-th distance:

$$d_k^* \le \hat{d}_k \tag{1}$$

This observation allows us to construct a search radius based on the *observed* HNSW results that guarantees the inclusion of the *unknown* true neighbors as per Figure 2.

Theorem 4.1 (Stretch-bounded containment of true top-k). *Given a query q, let $x_{(1)}$ be the nearest candidate and $\hat{d}_k$ be the distance to the k-th candidate returned by HNSW. Every point in the true top-k set $N_k^*(q)$ lies inside the graph ball of radius $t(\hat{d}_1 + \hat{d}_k)$ centered at $x_{(1)}$.*

Formally, for every $x^* \in N_k^*(q)$:

$$d_{G_0}(x_{(1)}, x^*) \le t(\hat{d}_1 + \hat{d}_k)$$

Consequently:

$$N_k^*(q) \subseteq \{x \in \mathcal{X} : d_{G_0}(x_{(1)}, x) \le t(\hat{d}_1 + \hat{d}_k)\}$$

Proof. Let x^* be any point in the true set $N_k^*(q)$. By definition of the true k-th distance and Equation 1:

$$dist(q, x^*) \le d_k^* \le \hat{d}_k \tag{2}$$

Using the triangle inequality in the metric space:

$$dist(x_{(1)}, x^*) \le dist(x_{(1)}, q) + dist(q, x^*) \tag{3}$$

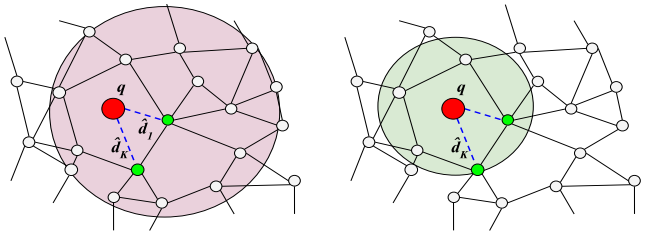

Figure 2: Stretch Bound Expansion. On the left, SBE-NN expands from the first NN of q a radius $\hat{d}_1 + \hat{d}_k$. On the right, SBE-Q expands a radius $\hat{d}_k$ from q

Substituting known variables $dist(x_{(1)}, q) = \hat{d}_1$ and the bound from Equation 2:

$$dist(x_{(1)}, x^*) \leq \hat{d}_1 + \hat{d}_k$$

This represents the maximum possible metric distance between the entry point $x_{(1)}$ and any true answer x^*. This worst-case occurs when q lies strictly between $x_{(1)}$ and x^* (collinear).

Since G_0 is a t-spanner, the graph distance is bounded by t times the metric distance:

$$d_{G_0}(x_{(1)}, x^*) \leq t \cdot dist(x_{(1)}, x^*) \tag{4}$$

Combining these inequalities yields:

$$d_{G_0}(x_{(1)}, x^*) \leq t(\hat{d}_1 + \hat{d}_k) \tag{5}$$

As x^* was arbitrary, the inclusion holds for all true top-k points. □

Figure 2 (left) illustrates Dijkstra's search region as the ball with radius $t \cdot (\hat{d}_1 + \hat{d}_k)$ centered at the first nearest neighbor. Any embedded vector outside of the ball is pruned.

4.2 Stretch-Bounded Expansion from Query (SBE-Q)

While SBE-NN (Section 4.1) guarantees exactness, the search radius relies on the triangle inequality to bridge the gap between the starting node $x_{(1)}$ and the query q. This introduces unnecessary "slack" in the search bound—specifically the term $\hat{d}_1 = dist(x_{(1)}, q)$. In high-dimensional spaces, even the nearest neighbor may be at a significant distance from q, inflating the search radius unnecessarily. To tighten this bound, we propose **SBE-Q**, which conceptually initiates the expansion directly from the query point q.

To formalize this, we treat the query q as a temporary node inserted into the graph G. Let $G' = (V', E')$ denote the graph augmented with q, where $V' = V \cup \{q\}$. Edges are established between q and the top-k computed candidates. This virtual insertion alters the graph topology. In Section 4.3, we demonstrate how to efficiently test the stretch factor of G'. Let t' be the stretch factor of the augmented graph G' (which may differ from t). We derive a bound depending solely on $\hat{d}_k$ and t', eliminating the $\hat{d}_1$ overhead (see Figure 2).

Theorem 4.2 (Stretch-Bounded Expansion from Query). *Let q be a query point virtually inserted into the graph G such that the augmented graph G' is a t'-spanner. Let $\hat{d}_k$ be the distance to the k-th candidate returned by HNSW. Every true top-k point lies inside the graph ball of radius $t' \cdot \hat{d}_k$ centered at q.*

Formally, for every $x^* \in N_k^*(q)$:

$$d_{G'}(q, x^*) \leq t' \cdot \hat{d}_k \tag{6}$$

Proof. Let x^* be any point in the true k-nearest neighbor set $N_k^*(q)$. By definition of the true set and the property that the HNSW k-th distance is an upper bound ($d_k^* \leq \hat{d}_k$):

$$dist(q, x^*) \leq d_k^* \leq \hat{d}_k \tag{7}$$

Since the augmented graph G' satisfies the t'-spanner property for the pair (q, x^*), the shortest path distance in G' is bounded by the metric distance scaled by t':

$$d_{G'}(q, x^*) \leq t' \cdot dist(q, x^*) \tag{8}$$

Combining Inequality (7) and Inequality (8), we obtain:

$$d_{G'}(q, x^*) \leq t' \cdot \hat{d}_k \tag{9}$$

Since this holds for any arbitrary $x^* \in N_k^*(q)$, the entire set of true nearest neighbors is contained within the graph ball $B_{G'}(q, t' \cdot \hat{d}_k)$. □

Remark 1. The SBE-Q bound $d_{G'}(q, x) \leq t' \cdot \hat{d}_k$ is strictly tighter than the SBE-NN bound $d_{G_0}(x_{(1)}, x) \leq t(\hat{d}_k + \hat{d}_1)$ whenever $t' \approx t$. By shifting the expansion center to q, we remove the penalty $\hat{d}_1$ (the distance from the query to the closest returned candidate). In Section 7.3, we empirically demonstrate that this approach significantly reduces the volume of the graph explored while maintaining probabilistic guarantees.

Figure 2 (right) illustrates Dijkstra's search region as the ball with radius $t \cdot \hat{d}_k$ centered at the query point q. The Dijkstra's expansion is smaller than the expansion obtained by starting the search at the first nearest neighbor (left panel), because we do not need to traverse the additional $t \cdot \hat{d}_1$ term again.

4.3 Incremental Stretch Factor Estimation

For an HNSW graph $G = (V, E)$ of realistic size, re-evaluating the stretch factor distribution from scratch after every "virtual" query insertion is computationally prohibitive. We seek to maintain the validity of the EVT bounds derived in Section 3 without incurring the cost of a full resample.

When we simulate the insertion of a query node q, the population of vertex pairs expands. To maintain a representative sample of tail extremes without full re-computation, we employ a reservoir-style update that probabilistically injects new pairs involving q while retaining existing sample values.

Algorithm: Incremental Reservoir Tail Patching. Because the insertion of q adds edges to the graph, the shortest path distance $d_G(u, v)$ for any existing pair (u, v) can only decrease or remain constant; it cannot increase. Therefore, the previously sampled extreme stretch values either remain accurate or act as conservative overestimates. We define the update probability p as the likelihood that a uniformly random pair in the augmented graph involves the new node q. If a newly sampled stretch factor involving q falls into the extreme right tail, it is injected into the pool of block maxima, and the GEV parameters (μ, σ, ξ) are efficiently refitted. Algorithm 1 presents the overall approach.

Algorithm 1 Incremental Reservoir Tail Patching for EVT Stretch Estimation

Require: Existing sample S_N partitioned into m blocks of size n, Graph size N, New query node z, Target confidence β, Previous stretch bound t
Ensure: Updated sample S_{N+1}, Updated GEV parameters $(\hat{\mu}, \hat{\sigma}, \hat{\xi})$, Updated return level t'

$p \leftarrow \frac{2}{N+1}$ {Probability a random pair involves z}
$M_{update} \leftarrow$ False {Flag to track if any block maximum requires recalculation}
for $i = 1$ **to** m **do**
  for $j = 1$ **to** n **do**
    Draw $u \sim \text{Uniform}(0, 1)$
    if $u < p$ **then**
      Select random node $v \in V_N$
      Compute $d_G(v, z)$ and $d_X(v, z)$
      $S_{i,j} \leftarrow \frac{d_G(v,z)}{d_X(v,z)}$ {Replace with new connection stretch}
      $M_{update} \leftarrow$ True
    else
      $S_{i,j} \leftarrow S_{i,j}$ {Retain old conservative value}
    end if
  end for
end for

if M_{update} is True **then**
  for $i = 1$ **to** m **do**
    $M_{N+1,i} \leftarrow \max(S_{i,1}, \ldots, S_{i,n})$
  end for
  $(\hat{\mu}, \hat{\sigma}, \hat{\xi}) \leftarrow \text{MLE}(M_{N+1})$ {Fit GEV distribution to the updated block maxima}
  $t' \leftarrow \hat{\mu} - \frac{\hat{\sigma}}{\hat{\xi}} \left[1 - (-\ln \beta)^{-\hat{\xi}}\right]$ {Compute new return level bound}
else
  $t' \leftarrow t$ {Retain previous bound t if reservoir was unchanged}
end if
return $S_{N+1}, (\hat{\mu}, \hat{\sigma}, \hat{\xi}), t'$

4.3.1 Theoretical Guarantees. We assert that the sample of extreme values updated by the Incremental Reservoir Tail Patching algorithm allows for a valid certification of the maximum stretch factor using the methodology of Theorem 3.2.

Theorem 4.3 (Conservative EVT Certification). *Let S_{ext} be the pool of extreme stretch values maintained during incremental insertion. Let $(\hat{\mu}, \hat{\sigma}, \hat{\xi})$ be the GEV parameters fitted to S_{ext}. For a confidence level β, the probability that the true maximum stretch factor of the augmented graph G' does not exceed the estimated return level t' is at least β.*

Proof. The validity of this bound relies on the monotonically non-increasing nature of graph distances under edge addition. For retained pairs not involving q, the stored stretch value S^{old} satisfies $S^{old} \geq S^{true}$. Therefore, the right tail of our sampled distribution stochastically dominates the true right tail of the augmented graph. Fitting the GEV distribution to these conservative maxima yields a return level t' that safely upper-bounds the true maximum stretch. □

4.3.2 Complexity Analysis. The computational efficiency of this incremental strategy is superior to full resampling. The number of distance computations required follows a Binomial distribution $K \sim B(n, p)$, where $p = \frac{2}{N+1}$. The expected number of computations is $E[K] = \frac{2n}{N+1}$.

For a typical configuration where the graph size N is large, the expected graph distance sampling cost is negligible. The primary additional overhead is the Maximum Likelihood Estimation (MLE) required to update the three GEV parameters (μ, σ, ξ). Because MLE is performed solely on the small subset of extreme tail values rather than the entire sample, it operates in $O(m)$ time (where m is the number of block maxima), effectively allowing for continuous certification with near-zero marginal cost per simulated insertion.

4.4 Filtered Stretch-Bounded Expansion from Query (F-SBE-Q)

The improved efficiency of SBE-Q (Section 4.2) can be directly extended to handle filtered search, where the goal is to retrieve the exact k-nearest neighbors that satisfy a condition P. Standard heuristic traversals fail in these scenarios when filters disconnect the "small world" topology. However, by leveraging the global connectivity of the underlying graph structure—regardless of the predicate status of intermediate nodes—we can guarantee exactness using the bounds derived from the approximate search results.

4.4.1 Problem Formulation. Let $P : \mathcal{X} \to \{0, 1\}$ be a boolean predicate function. We seek the set of k nearest neighbors $\mathcal{N}^*_{k,P}(q)$ such that for all $x \in \mathcal{N}^*_{k,P}(q)$, $P(x) = 1$. Let $d^*_{k,P}$ denote the distance to the true k-th nearest neighbor in the satisfying set:

$$d^*_{k,P} = \max_{x \in \mathcal{N}^*_{k,P}(q)} dist(q, x) \tag{10}$$

Let $\mathcal{H}_{found}$ be the set of candidates returned by the initial HNSW search (or any heuristic phase). We identify the subset of these candidates that satisfy the predicate P. If at least k such candidates exist, let $\hat{d}_{k,P}$ denote the distance to the k-th closest satisfying candidate found so far.

Similar to the unfiltered case, the distance to the k-th candidate found by any approximate method is an upper bound on the true k-th distance:

$$d^*_{k,P} \leq \hat{d}_{k,P} \tag{11}$$

4.4.2 Theoretical Guarantees. We extend Theorem 4.2 to the filtered case using the observable bound $\hat{d}_{k,P}$. The critical insight is that while the *targets* must satisfy P, the *path* taken in the graph G' to reach them need not. Thus, we rely on the spanner property of the full augmented graph G'.

Theorem 4.4 (Filtered SBE-Q). *Let q be a query point virtually inserted into the graph G such that the augmented graph G' is a t'-spanner. Let $\hat{d}_{k,P}$ be the distance to the k-th approximate neighbor satisfying predicate P (as returned by HNSW or updated dynamically). Then, every true filtered top-k point lies inside the graph ball of radius $t' \cdot \hat{d}_{k,P}$ centered at q.*

Formally, for every $x^* \in \mathcal{N}^*_{k,P}(q)$:

$$d_{G'}(q, x^*) \leq t' \cdot \hat{d}_{k,P} \tag{12}$$

Proof. Let x^* be any point in the true filtered set $\mathcal{N}^*_{k,P}(q)$. By definition of the true set and the upper bound property of the approximate results:

$$dist(q, x^*) \leq d^*_{k,P} \leq \hat{d}_{k,P} \tag{13}$$

Since the graph G' is a t'-spanner over the full set of vertices $V \cup \{q\}$, there exists a path between q and x^* in G' such that:

$$d_{G'}(q, x^*) \leq t' \cdot dist(q, x^*) \tag{14}$$

Substituting the metric bound:

$$d_{G'}(q, x^*) \leq t' \cdot \hat{d}_{k,P} \tag{15}$$

Algorithm 2 Filtered Stretch-Bounded Expansion from Query (F-SBE-Q)

Require: Query q, Predicate P, Graph G', Stretch t', Initial Candidates $\mathcal{H}_{found}$
Ensure: Exact filtered set $\mathcal{N}^*_{k,P}(q)$

Initialize:
$Q_{visit} \leftarrow$ Min-Priority Queue ordered by graph distance $d_{G'}$
$R_{topK} \leftarrow$ Max-Priority Queue of size k (stores pairs $(dist(q,u), u)$ where $P(u) = 1$)
$Visited \leftarrow \{q\}$
{Seed with initial heuristic results that satisfy P to establish $\hat{d}_{k,P}$}
for $u \in \mathcal{H}_{found}$ **do**
  if $P(u)$ is True **then**
    R_{topK}.push$(dist(q,u), u)$
  end if
end for
Q_{visit}.push$(0, q)$
while Q_{visit} is not empty **do**
  $(d_{graph}, u) \leftarrow Q_{visit}$.pop()
  {Dynamic Termination Condition using $\hat{d}_{k,P}$}
  $\hat{d}_{k,P} \leftarrow R_{topK}$.max_dist() {Returns ∞ if $< k$ found}
  if $d_{graph} > t' \cdot \hat{d}_{k,P}$ **then**
    break {Guaranteed to have found all true filtered neighbors}
  end if
  {Expand Neighbors}
  for each neighbor v of u in G' **do**
    if $v \notin Visited$ **then**
      $Visited$.add(v)
      $new\_d_{graph} \leftarrow d_{graph} + dist(u,v)$
      {Check Metric Distance & Predicate}
      $d_{metric} \leftarrow dist(q,v)$
      if $P(v)$ is True **and** $d_{metric} < R_{topK}$.max_dist() **then**
        R_{topK}.push(d_{metric}, v) {Updates $\hat{d}_{k,P}$ (tightens bound)}
      end if
      Q_{visit}.push(new_d_{graph}, v)
    end if
  end for
end while
return R_{topK}

Thus, the search radius defined by the approximate results is sufficient to encompass all true answers. □

4.4.3 Algorithm. The expansion algorithm (Algorithm 2) proceeds by traversing the graph G' starting from q. Crucially, to preserve the spanner property t', the traversal visits nodes regardless of whether they satisfy P. The predicate $P(v)$ is checked only during the candidate collection phase to update the dynamic bound $\hat{d}_{k,P}$. While Algorithm 2 presents the logic for predicate handling, in practice, this traversal is further optimized using the Metric Bound Verification techniques we introduce in Section 5.

5 METRIC BOUND VERIFICATION

While the Stretch-Bounded Expansion (SBE-Q and F-SBE-Q) described in Sections 4.2 and 4.4 provides a theoretical containment guarantee for the true k-nearest neighbors, naively evaluating the metric distance $dist(q,v)$ for every node v within the search radius $t' \cdot d_k$ can be computationally expensive. To bridge the gap between theoretical exactness and practical latency, we introduce Metric Bound Verification (MBV).

MBV is a pruning mechanism integrated directly into the graph traversal. It exploits the triangular inequality inherent in the metric space to discard candidates *within* the search radius without performing explicit distance computations. By maintaining recursive lower bounds on the distance from the query to the current node, we can rigorously certify that specific sub-branches of the graph cannot contain better candidates than those already found.

These pruning mechanisms are universal; they apply equally to the unconstrained SBE-Q search and the filtered F-SBE-Q search, as they rely solely on metric space properties independent of node attributes.

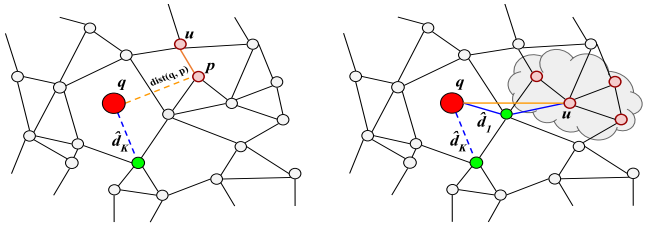

Figure 3: Metric Bounded Verification. On the left, Recursive Lower Bound Pruning is shown: the parent p was pruned, and the lower bound LB(u) was then used to eliminate u. On the right, Elliptical Search Space Pruning is shown, where node u is pruned along with its descendants (Equation 17).

5.1 Recursive Lower Bound Pruning

The core of MBV relies on propagating lower bounds down the search tree (the path taken by Dijkstra's algorithm from the virtual query node q). For any node u visited during the expansion, let p be its parent in the search trace (i.e., the node that added u to the priority queue). By the triangle inequality, the metric distance $dist(q,u)$ is lower-bounded by the distance to the parent minus the edge weight connecting them:

$$dist(q,u) \geq dist(q,p) - dist(p,u) \tag{16}$$

Since the edge weight $w(p,u)$ in the graph corresponds to the metric distance between p and u, we can derive a recursive lower bound $LB(u)$. This bound depends on whether the parent p was fully evaluated or merely pruned (where only a lower bound is known).

- **Evaluated Parent:** If $dist(q,p)$ has been computed (note if this distance was in the Search Trace $\mathcal{T}(q)$, as per section 2, it will be re-used), then: $LB(u) = dist(q,p) - w(p,u)$
- **Pruned Parent:** If p was pruned based on its own lower bound $LB(p)$, we propagate the uncertainty: $LB(u) = LB(p) - w(p,u)$

Let $\hat{d}_k$ be the distance to the current k-th nearest neighbor found so far (initially from HNSW, then updated dynamically). If $LB(u) > \hat{d}_k$, we can certify that u is strictly worse than the current candidate set and cannot improve the result. Consequently, we mark u as PRUNED and skip the expensive $dist(q,u)$ calculation (Figure 3). Crucially, we still explore u's neighbors, as the graph topology may lead back toward q, but we do so carrying the derived lower bound.

5.2 Elliptical Search Space Pruning

Beyond avoiding distance computations, we can also prune the search space itself using an "Elliptical Intersection" check. If the exact distance $dist(q,u)$ is computed, we can determine if the entire

branch stemming from u is effectively dead. A node u can be discarded—preventing the addition of its neighbors to the queue—if the path through u cannot possibly reach a point closer than the current k-th neighbor bound $\hat{d}_k$ within the remaining search budget.

Formally, if $d_{G'}(q,u)$ is the graph distance traveled so far and $R_{search} = t' \cdot \hat{d}_k$ is the current stretch-bounded radius, any descendant v must satisfy $d_{G'}(q,v) \le R_{search}$. This implies the "remaining fuel" from u is $R_{search} - d_{G'}(q,u)$. If the metric distance $dist(q,u)$ exceeds this remaining capacity plus the target distance $\hat{d}_k$, no descendant can qualify. The pruning condition is (Figure 3):

$$d_{G'}(q,u) + dist(q,u) > t' \cdot \hat{d}_k + \hat{d}_k \tag{17}$$

This check tightens the search region from a simple graph ball to the intersection of the graph ball and the underlying metric constraints.

5.3 Algorithm

The integrated procedure, SBE-Q with Metric Bound Verification, is detailed in Algorithm 3. It replaces the naive expansion phases, ensuring that expensive metric evaluations are minimized while maintaining the probabilistic guarantees of exactness derived in Theorem 4.2.

This algorithm upgrades the heuristic search of an HNSW graph into an exact retrieval engine by rigorously bounding the search space and pruning unnecessary calculations. It operates through two integrated mechanisms:

Stretch-Bounded Expansion (SBE):. The search treats the query q as a virtual node and expands strictly within a dynamic radius $R_{\text{prune}} = t' \cdot \hat{d}_k$. This radius is derived from the graph's stretch factor t' and the current best-known k-th distance $\hat{d}_k$. Since the true k-th distance d_k^* satisfies $d_k^* \le \hat{d}_k$, this radius guarantees that all true k-nearest neighbors are contained within the boundary. As better candidates are discovered, $\hat{d}_k$ decreases, automatically shrinking the search radius. Specifically:

- **Discovery of Better Candidates:** As the expansion traverses the graph, it computes the exact metric distance d_{true} for a visited node u. If $d_{true} < \hat{d}_k$, u improves upon the current top-k set.
- **The Update Event:** The algorithm inserts u into the priority queue of results (R_{topK}) and removes the previous furthest candidate. Consequently, $\hat{d}_k$ immediately decreases.
- **Tightening the Radius:** This reduction in $\hat{d}_k$ triggers an immediate recalculation of $R_{prune} = t' \cdot \hat{d}_k$. The algorithm effectively "pulls in" the search horizon, pruning pending nodes in the queue that are now outside the new, tighter boundary.

Metric Bound Verification (MBV):. To avoid expensive metric distance computations for every visited node, the algorithm applies two pruning checks:

(1) **Recursive Lower Bound Pruning:** It utilizes the triangle inequality to propagate lower bounds from parent nodes to their children. If a node's lower bound distance $LB(u)$ exceeds the current $\hat{d}_k$, the node is discarded without performing the expensive distance calculation.

(2) **Elliptical Pruning:** It evaluates if a graph branch is "dead" by checking if the path through a node can possibly reach a better candidate within the remaining search budget. If $d_{G'}(q,u) + dist(q,u) > (t'+1) \cdot \hat{d}_k$, the entire branch is pruned.

Together, these techniques guarantee the retrieval of the exact k-nearest neighbors while maintaining efficiency by aggressively filtering out unpromising candidates. Figure 3 (right) presents the Elliptical Search Space Pruning technique where the node u is far enough from q by graph distance (path shown in blue), such that any descendant of u (shown by the red cloud) in the shortest-path tree will not be within the top-k as they are far away in vector space as well (shown by the orange line) and thus, can be pruned.

6 STATISTICAL CERTIFICATION VIA CONFORMAL RISK CONTROL

We frame the "Certify" phase of our "Certify-then-Rectify" workflow as a distribution-free selective prediction problem. By leveraging Conformal Risk Control (CRC) [3, 4] and Learn-then-Test (LTT) [5] procedures, we construct a rigorous statistical certifier. This certifier acts as a gatekeeper, dynamically deciding whether to trust the fast, approximate candidates returned by HNSW, or to escalate to the exact, but computationally heavier, exact recovery procedures (SBE-Q and MBV).

6.1 Problem Setup and Certifiability Score

Let $\mathcal{N}_k(q)$ represent the approximate top-k candidates returned by the vanilla HNSW greedy traversal. Let $\mathcal{N}_k^*(q)$ represent the true k-nearest neighbors obtained via an exact oracle (in our framework, this oracle is SBE-Q coupled with MBV). We define the per-query recall as:

$$Recall(q) = \frac{|\mathcal{N}_k(q) \cap \mathcal{N}_k^*(q)|}{k} \tag{18}$$

Given a target recall threshold $\tau \in (0,1]$, we call a top-k result set "compliant" to τ if $Recall(q) \ge \tau$. Given τ and a target risk level $\alpha \in (0,1)$, our goal is to build a certifier $C(q)$ that outputs "accept" only when it can statistically guarantee that the top-k result set is compliant with τ.

To achieve this, we define a score $s(q)$ from HNSW's internal search trace. We use a compact feature vector $z(q)$ that captures (i) distance geometry, (ii) search reach, and (iii) path stability:

$$z(q) = \left[\hat{d}_1, \dots \hat{d}_{100},\ \tilde{d}_1, \dots, \tilde{d}_{10}, |\mathcal{T}(q)|,\ n_{\text{rev}}(q),\ \Delta_{\text{rev}}(q)\right].$$

$\hat{d}_1, \dots, \hat{d}_{100}$ are the top-100 returned distances, and in the case $k < 100$, we pad with 0 entries. $\tilde{d}_1, \dots, \tilde{d}_{10}$ are the remaining top-10 candidate-frontier distances at termination. These features capture distance geometry. Following previous work [11], we capture the search reach with $|\mathcal{T}(q)|$, where $\mathcal{T}(q)$ is the search trace defined in Section 2. We use *distance-reversal* statistics for path stability: $n_{\text{rev}}(q)$ is the number of direction reversals in successive expansion distances, and $\Delta_{\text{rev}}(q)$ is their cumulative magnitude. More precisely, let

$$d_1^{\text{pop}}, d_2^{\text{pop}}, \dots, d_T^{\text{pop}}$$

denote the sequence of metric distances from q of the nodes popped from the bottom-layer HNSW candidate queue. A *distance reversal*

occurs at index $j \in \{2, \ldots, T-1\}$ whenever the local direction of this sequence changes, i.e.,

$$\left(d_j^{\text{pop}} - d_{j-1}^{\text{pop}}\right)\left(d_{j+1}^{\text{pop}} - d_j^{\text{pop}}\right) < 0.$$

Equivalently, the popped distances are locally non-monotone; for example, 100, 50, 120 and 50, 120, 80 each contain a reversal. Let $r_j(q)$ indicate if a local reversal occurs at position j. We define $n_{\text{rev}}(q)=\sum_{j=2}^{T-1} r_j(q)$ as the number of reversals, and $\Delta_{\text{rev}}(q)$ as the sum of the adjacent jump magnitudes (ie. the two factors of the product in the above equation) over all positions j with $r_j(q)$=1. These features quantify instability in the HNSW traversal order: larger values indicate that the search path is less geometrically consistent. Our feature-ablation results indicate that distance geometry contributes most of the score separability, while path stability provide complementary gains in harder regimes: removing distance features reduces AUROC by $0.009 \sim 0.055$ across datasets. Removing reversal features has a smaller effect but still non-negligible (up to ΔAUROC =0.017). This supports using the full vector for cross-dataset robustness. The feature vector is then mapped through logistic regression trained on a calibration set. Crucially, the statistical guarantee is independent of the model's predictive quality; a poor fit simply yields a more conservative certifier.

6.2 Guaranteeing Expected Shortfall via CRC

To guarantee the average quality of the certified results without making underlying distributional assumptions about the metric space, we utilize the Conformal Risk Control (CRC) framework. We define the recall shortfall as our per-query loss function:

$$l(q) = \max(0, \tau - Recall(q)) \tag{19}$$

Given a calibration set $\mathcal{D}_{cal} = \{(q_i, \mathcal{N}_k^*(q_i))\}_{i=1}^n$ drawn i.i.d. from an unknown query distribution $\mathcal{P}$, our procedure is as follows:

(1) Compute the score $s_i = s(q_i)$ and the loss l_i for each calibration query, sort them in descending order of score.
(2) For any candidate threshold θ, define the empirical risk as:

$$\hat{R}(\theta) = \frac{\sum_{i=1}^n l_i \cdot \mathbb{1}[s_i \geq \theta]}{\sum_{i=1}^n \mathbb{1}[s_i \geq \theta] + 1} \tag{20}$$

(3) Select the optimal $\hat{\theta}$ that bounds this empirical risk:

$$\hat{\theta} = \inf\left\{\theta : \hat{R}(\theta) \leq \alpha(1-\tau) \cdot \frac{n}{n+1}\right\} \tag{21}$$

Theorem 6.1 (CRC Guarantee for HNSW Recall). *If the calibration queries $q_1, \ldots, q_n$ are i.i.d. from $\mathcal{P}$, then for a new test query $Q \sim \mathcal{P}$, the certifier $C(Q) = \mathbb{1}[s(Q) \geq \hat{\theta}]$ satisfies:*

$$\mathbb{E}[l(Q) \cdot \mathbb{1}[s(Q) \geq \hat{\theta}]] \leq \alpha(1-\tau) \tag{22}$$

This guarantee is exact in a finite-sample sense and requires no knowledge of HNSW's internal workings.

6.3 Pointwise Guarantees via Learn-then-Test (LTT)

If a strict, per-query probabilistic guarantee is required, we deploy the Learn-then-Test (LTT) framework. LTT controls the certifier's precision, ensuring that $\mathbb{P}_{q\sim\mathcal{P}}[Recall(q) \geq \tau \mid s(q) \geq \hat{\theta}] \geq 1-\epsilon$ with a confidence level of $1-\alpha$.

We split the calibration set $\mathcal{D}_{cal}$ into a training set $\mathcal{D}_{tr}$ and a test set $\mathcal{D}_{te}$. From $\mathcal{D}_{tr}$, we define a finite set of candidate thresholds Θ. For each candidate threshold $\theta \in \Theta$ evaluated on $\mathcal{D}_{te}$, we test the null hypothesis $H_0^\theta : \mathbb{P}[Recall(q) < \tau \mid s(q) \geq \theta] > \epsilon$.

Let m_θ denote the number of certified queries and x_θ denote the subset of those queries that failed to meet the target recall. We compute the one-sided binomial p-value:

$$\text{p-value} = \sum_{j=0}^{x_\theta} \binom{m_\theta}{j} \epsilon^j (1-\epsilon)^{m_\theta - j} \tag{23}$$

To account for multiple testing over Θ, we apply a Bonferroni correction, rejecting H_0^θ only if p-value $\leq \frac{\alpha}{|\Theta|}$. The final threshold is selected as:

$$\hat{\theta} = \max\{\theta \in \Theta : H_0^\theta \text{ is rejected}\} \tag{24}$$

6.4 The Query-Time Decision Rule

With $\hat{\theta}$ calibrated, the online query processing acts as a simple, two-step decision process:

Require: Query q
1: Run HNSW to obtain $\mathcal{N}_k(q)$ and compute the score $s(q)$.
2: **if** $s(q) \geq \hat{\theta}$ **then**
3: **CERTIFY:** Return $\mathcal{N}_k(q)$.
4: **else**
5: **ESCALATE:** Run the exact oracle (SBE-Q and MBV) and return $\mathcal{N}_k^*(q)$.
6: **end if**

7 EXPERIMENTS

7.1 Datasets and Implementation Setup

Dataset	Description	d	#queries
SIFT1M	Image feature transform vectors	128	10k
GIST1M	GIST image descriptors	960	1k
DEEP1M	Subset of deep image features	96	10k

Table 1: Parameters Of Datasets Used

We augment `nmslib/hnswlib`[1] in C++ to implement the proposed rectification algorithms, namely SBE-NN and SBE-Q, together with the CTR pipeline built on top of HNSW. We evaluate the system on three standard benchmark datasets: SIFT1M and GIST1M[2], and DEEP1M[3], all under the L_2 metric. Dataset statistics are summarized in Table 1.

We vary M, efc, efs, and k and examine both the cost of exact rectification and the behavior of the certification stage under different HNSW operating points. We therefore use a fixed 90/10 calibration/test split of the number of query vectors in each dataset (see the last column of Table 1).

For experiments involving a target recall threshold τ, we choose τ relative to the baseline HNSW operating point of the corresponding dataset or configuration. This avoids evaluating clearly unattainable targets for a weak baseline and makes the certification–runtime trade-off more meaningful in practice.

7.2 Validating Stretch Estimation via Level-2 Ground Truth

We validate our stochastic stretch estimation method per Section 3. To do this, we compare the estimated stretch to the true maximum

[1] https://github.com/nmslib/hnswlib
[2] Both SIFT1M and GIST1M from: http://corpus-texmex.irisa.fr/
[3] https://research.yandex.com/blog/benchmarks-for-billion-scale-similarity-search

Algorithm 3 Stretch-Bounded Expansion with MBV

Require: Query q, Graph G', Stretch t', Initial Candidates $\mathcal{H}_{found}$
Ensure: Exact k-nearest neighbor set $\mathcal{N}_k^*(q)$

Initialize:
$R_{topK} \leftarrow$ Max-Priority Queue of size k (seeded with $\mathcal{H}_{found}$)
$Q_{visit} \leftarrow$ Min-Priority Queue ordered by graph distance $d_{G'}$
$State \leftarrow$ Map (node_id: {status, val})
Set initial bound based on HNSW results:
$\hat{d}_k \leftarrow R_{topK}.max\_dist()$
$R_{prune} \leftarrow t' \cdot \hat{d}_k$
$Q_{visit}.push(0, q, \text{null})$ {(distance, node, parent)}
$State[q] \leftarrow$ {status: EVALUATED, val: 0}
while Q_{visit} is not empty **do**
  $(d_{graph}, u, p) \leftarrow Q_{visit}.pop()$
  if $d_{graph} > R_{prune}$ **then**
    break {Dynamic Termination}
  end if
  Calculate Lower Bound (MBV):
  $LB_u \leftarrow -\infty$
  if $p \neq$ null **then**
    $w \leftarrow$ edge_weight(p, u)
    $LB_u \leftarrow State[p].val - w$ {Propagate distance or bound}
  end if
  Pruning Check & Metric Computation:
  if $LB_u > \hat{d}_k$ **then**
    $State[u] \leftarrow$ {status: PRUNED, val: LB_u} {Skip expensive distance call}
  else
    $d_{true} \leftarrow dist(q, u)$ {Expensive computation}
    $State[u] \leftarrow$ {status: EVALUATED, val: d_{true}}
    Update Result Set:
    if $d_{true} < \hat{d}_k$ **then**
      $R_{topK}.push(d_{true}, u)$
      $\hat{d}_k \leftarrow R_{topK}.max\_dist()$
      $R_{prune} \leftarrow t' \cdot \hat{d}_k$ {Dynamically tighten search radius}
    end if
    Elliptical Pruning (Look-ahead):
    if $d_{graph} + d_{true} > R_{prune} + \hat{d}_k$ **then**
      continue {Entire branch pruned}
    end if
  end if
  Expand Neighbors:
  for all neighbor v of u in G' **do**
    if $v \notin State$ **then**
      $new\_d \leftarrow d_{graph} +$ edge_weight(u, v)
      $Q_{visit}.push(new\_d, v, u)$
    end if
  end for
end while
return R_{topK}

stretch computed via brute force on the level-2 layer of HNSW, which encompasses fewer nodes and makes the ground-truth computation manageable[4]. To validate estimated stretch we: (i) compute the exact level-2 true maximum stretch for a given (M, ef_c); (ii) run stochastic stretch sampling with increasing sample size, conduct 100 bootstrap trials at each sample point and obtain the maximum estimated stretch at each sample size.

Figure 4 presents the estimation on SIFT1M, DEEP1M, and GIST1M (M=32, ef_c=100, β=0.995 for all three). The solid line shows our estimated t, and the dotted line is the true t_{max} of the graph. At very small sample sizes, fitted t can be unstable and occasionally overestimate due to small-sample EVT fitting artifacts. As sample size increases, the fitted values stabilize and saturate with the true t_{max} as upper envelope (with a small error < 3% in all cases).

[4]Vanilla HNSW has no guarantee in connectivity (see also [42]). Our stretch calculation is therefore conducted only for reachable nodes, despite unreachable nodes being rare (3 *in degree*=0 nodes in our SIFT1M M=16, efc=100 graph).

This demonstrates that the sampling strategy is reliable enough to transfer to the base layer. For base-layer operation (where true max is unknown), we therefore adopt the same strategy and increase sampling until the fitted stretch statistics saturate; the saturated value is then used as the operational stretch t^*.

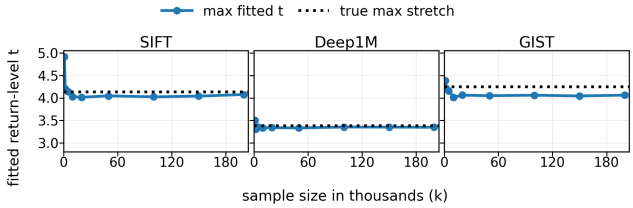

Figure 4: Level-2 validation of EVT stretch estimation.

7.3 SBE-Q and SBE-NN Comparison

We compare the cost of SBE-Q and SBE-NN by reporting the fraction of base-layer nodes expanded during the Dijkstra phase. This experiment is independent of the final certification stage, so no target recall is enforced during testing[5]. Table 2 reports results on SIFT1M, GIST1M, and DEEP1M for different M, ef_c, and k, with $ef_s = 100$ throughout.

Both methods use the same calibrated stretch t per configuration, where t is obtained from the validated stochastic-EVT protocol described in Section 7.2 (increasing sample size until fitted stretch saturates). Thus, this comparison isolates expansion-rule differences, not differences in stretch tuning.

Our main observation is **SBE-Q expands** $2.2 \sim 51$ **times fewer nodes than SBE-NN** across all datasets and parameter settings. This matches the analysis in Section 4. SBE-Q expands from the query using the tighter radius $t\hat{d}_k$, whereas SBE-NN expands from the first returned neighbor using the looser radius $t(\hat{d}_1 + \hat{d}_k)$. Removing the extra $\hat{d}_1$ term substantially shrinks the graph region explored during rectification. We also observe that increasing M and ef_c generally reduces the estimated stretch, indicating that graph shortest-path distances better approximate the underlying metric distances. This trend is explored more comprehensively in Section 7.3 of our technical report [6]. In most cases, this reduces the Dijkstra expansion induced by the stretch bound and lowers the rectification cost, but this is not always monotone: a denser graph can reduce the graph distance needed to reach intermediate nodes, so Dijkstra may reach them at shorter graph distance and still expand further before termination, even under a smaller radius bound. Similar observations hold for higher ef_c values. In contrast, increasing k increases $\hat{d}_k$, enlarges the stretch-bounded radius, and therefore raises the number of expanded nodes for both methods. Overall, the tighter SBE-Q bound translates into a substantial empirical improvement in rectification efficiency.

7.3.1 Filtered Search. SBE-Q scales naturally to filtered ANN search (Section 4.4). To demonstrate this, we extend ACORN [42], a representative filtered-search method. ACORN is a variant of HNSW and explores 2-hop neighbors at each step, adding extra edges through the navigability parameter γ. The results in Table 3 depict SBE-Q on ACORN. In general, the trends are similar to those with HNSW

[5]The expansion takes place in every query and returns the true set of neighbors.

Dataset	Metric	$M=16$, $ef_c=100$			$M=16$, $ef_c=200$			$M=32$, $ef_c=100$			$M=32$, $ef_c=200$		
	k	10	50	100	10	50	100	10	50	100	10	50	100
SIFT1M	t	4.059	4.059	4.059	3.853	3.853	3.853	3.657	3.657	3.657	3.601	3.601	3.601
	SBE-Q (% of 1M)	5.60	14.07	18.66	4.04	11.38	15.82	3.40	10.13	14.22	3.62	10.37	14.67
	SBE-NN (% of 1M)	48.95	50.54	51.41	48.16	49.66	50.47	47.42	48.59	49.21	48.47	50.00	50.83
DEEP1M	t	4.251	4.251	4.251	3.748	3.748	3.748	4.067	4.067	4.067	3.867	3.867	3.867
	SBE-Q (% of 1M)	3.82	12.60	18.89	1.39	5.96	9.82	4.71	14.32	21.03	3.65	12.64	19.07
	SBE-NN (% of 1M)	81.03	84.75	86.52	71.36	76.12	78.40	83.43	86.86	88.51	83.08	86.53	88.19
GIST1M	t	4.470	4.470	4.470	4.119	4.119	4.119	3.930	3.930	3.930	3.646	3.646	3.646
	SBE-Q (% of 1M)	15.67	31.42	38.75	10.04	23.54	30.90	9.42	22.89	30.39	6.66	17.26	24.07
	SBE-NN (% of 1M)	84.91	86.08	86.63	83.83	85.15	85.77	85.10	86.40	87.01	83.95	85.26	85.87

Table 2: Expanded-node comparison across SIFT1M, DEEP1M, and GIST1M.

Dataset	Metric	$M=16$, $ef_c=200$, $k=10$				$M=16$, $ef_c=200$, $k=100$				$M=32$, $ef_c=200$, $k=10$				$M=32$, $ef_c=200$, $k=100$			
	σ	1	5	10	25	1	5	10	25	1	5	10	25	1	5	10	25
SIFT1M	t	3.853	3.853	3.853	3.853	3.853	3.853	3.853	3.853	3.601	3.601	3.601	3.601	3.601	3.601	3.601	3.601
	SBE-Q (% of 1M)	67.50	5.19	4.19	3.24	75.42	17.56	14.80	12.08	10.62	6.84	5.30	4.13	30.03	20.79	17.51	14.71
DEEP1M	t	3.748	3.748	3.748	3.748	3.748	3.748	3.748	3.748	3.867	3.867	3.867	3.867	3.867	3.867	3.867	3.867
	SBE-Q (% of 1M)	11.25	6.11	4.95	4.28	49.29	22.85	18.62	14.88	6.41	3.30	2.61	2.24	34.10	14.53	11.44	9.00
GIST1M	t	4.119	4.119	4.119	4.119	4.119	4.119	4.119	4.119	3.646	3.646	3.646	3.646	3.646	3.646	3.646	3.646
	SBE-Q (% of 1M)	18.48	18.32	11.52	10.24	46.34	34.98	27.18	23.78	18.40	13.53	12.35	10.65	44.48	33.54	30.28	26.62

Table 3: Expanded-node comparison on SIFT1M, DEEP1M, GIST1M (Filtered Search)

(Table 2). We introduce a selectivity parameter σ, which depicts the percentage of database vectors satisfying a predefined query predicate. As shown in Table 3, decreasing σ (i.e., higher selectivity) results in an increase in the number of nodes explored by SBE-Q, while recall is perfect across all cases (subject to estimation of t). More selective predicates naturally increase d_k, the top-k filtered results lie farther from q, which in turn enlarges the search radius. It is known [42] that ACORN exhibits low recall with selective predicates and our framework resolves this.

7.4 Recall Target Compliance of Conformal Parameters

We next evaluate how the conformal parameters affect the final recall–runtime trade-off of CTR via the CRC/LTT. Rather than reporting average recall, we utilize per-query result-set compliance from Section 6.1 and report the *compliance rate* over a set of queries, which is the fraction of queries for which $Recall(q) \geq \tau$. Compliance rate is based on the result set after the rectification step, if certification fails.

In these experiments, we fix the underlying HNSW parameters to M=16, ef_c=100, and ef_s=100. For the α-sweeps in Figure 5(a)–(f), we fix the target recall threshold to τ=0.90 for SIFT1M and DEEP1M, and τ=0.75 for GIST1M per our reasoning in Section 7.1. Panels (a)–(c) present the CRC α-sweeps on SIFT1M, GIST1M, and DEEP1M, respectively, while panels (d)–(f) demonstrate the corresponding LTT α-sweeps on the same three datasets. For the τ-sweeps in Figure 6, we keep the same HNSW configuration and vary the target recall threshold τ. In all plots, the x-axis reports runtime as a multiple of the runtime of HNSW (without our enhancements) under the same configuration. The plots exhibit a consistent pattern for both CRC and LTT: stricter certification settings with small α lead to a higher fraction of queries satisfying $Recall(q) \geq \tau$, but that triggers rectification more often and therefore increases runtime. Conversely, larger values of α make direct certification easier, which reduces runtime but also lowers the fraction of outputs satisfying the target recall threshold. Figures 5(a)–(c), present the operating points obtained by varying α; the numeric annotation next to each blue point is the corresponding value of α. As α decreases, fewer HNSW outputs are certified directly and more queries are rectified. As a result, the fraction of queries satisfying the target recall threshold increases, but runtime also increases. The corresponding τ-sweeps for CRC are presented in Figures 6(d)–(f). Compared with the compliance rate graphs of HNSW in Figures 6(a)–(c), they reveal a complementary effect: as the target recall threshold becomes higher, HNSW answers are less often compliant with τ, while the CTR pipeline maintains a substantially higher compliance through rectification.

LTT exhibits the same qualitative trend in Figures 5(d)–(f) and 6(g)–(i). Decreasing α makes certification harder, so fewer queries are certified directly and more are rectified. This consequently leads to increase in both the compliance rate and runtime; larger α has the opposite effect. Notably, the small α values that appear in LTT are largely an inherent consequence of the test itself: the null benchmark is the boundary case with x_θ failures among m_θ certified queries under a Binomial(m_θ, ϵ) model, and the corresponding one-sided tail probability can become extremely small as soon as the observed failure count falls below the nominal level ϵm_θ. Thus, the practically useful range of α in LTT is naturally shifted toward smaller values; this is a property of the testing mechanism rather than an unusually aggressive confidence requirement.

Figure 7 further illustrates how the LTT threshold θ varies jointly with α and ϵ. The qualitative trend of θ varying with α and ϵ is consistent. In particular, along a fixed-ϵ slice of the surface, decreasing α increases the selected threshold θ. Since LTT certifies a query only when its score exceeds θ, a larger threshold makes certification more selective: fewer queries are accepted directly, more queries are sent to rectification, and the final compliance rate increases at the cost of higher latency. Conversely, for the same fixed ϵ, increasing α lowers θ, which certifies more queries directly and therefore reduces runtime, but also lowers compliance. To visually illustrate this trade-off, the black contour lines in Figure 7 depict constant-ϵ cross-sections of the threshold surface: ϵ=0.4 for SIFT1M and DEEP1M, and ϵ=0.7 for GIST1M. Each such cross-section isolates the one-dimensional family of operating points obtained by varying α while holding ϵ fixed, thereby making clear how the resulting

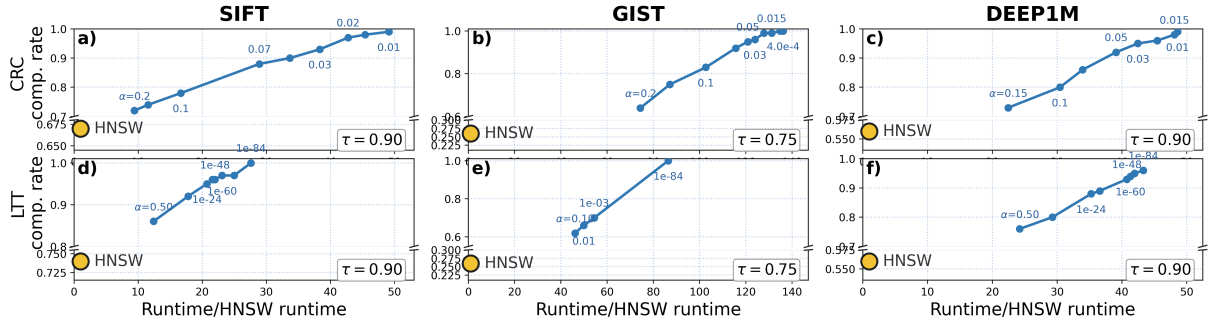

Figure 5: α-sweeps on three datasets. Blue line = compliance rate as α is varied; x-axis is in multiples of HNSW runtime.

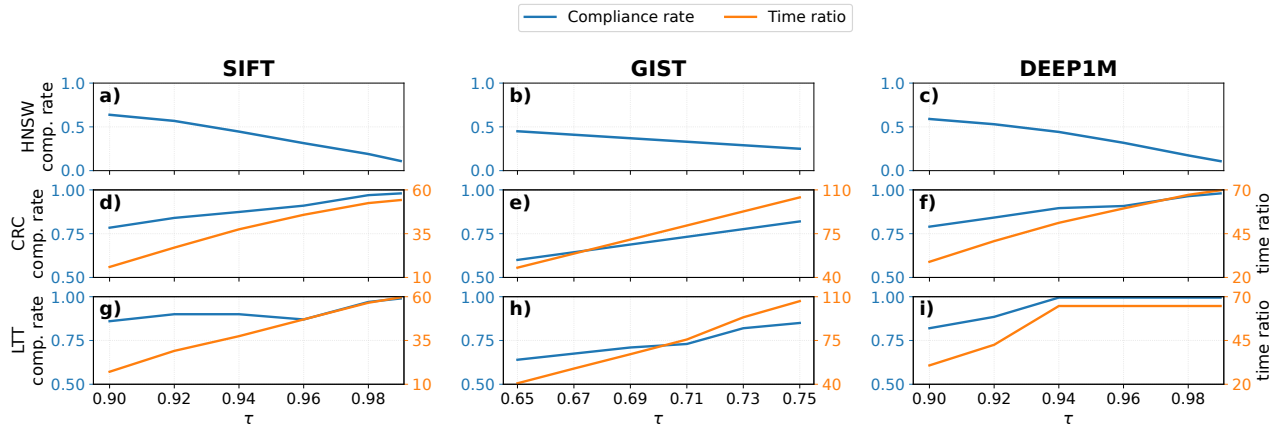

Figure 6: τ-sweeps on three datasets. Primary scale = compliance rate; secondary scale = time in multiples of HNSW runtime.

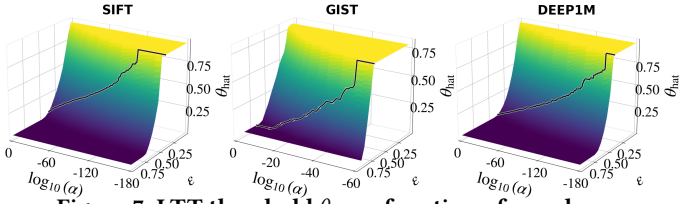

Figure 7: LTT threshold θ as a function of α and ϵ.

change in θ governs the latency–compliance trade-off described above. The figure also suggests a practical tuning strategy: first select ϵ so as to avoid the broad plateau region of θ (the yellow region, where variation in ϵ has relatively little effect on the threshold), and then vary α to choose the desired operating point along the corresponding constant-ϵ trade-off curve.

We next study how the HNSW operating point affects the latency–compliance trade-off. In Figure 8, the blue and red curves correspond to ef_{search}=100, while the green and brown curves correspond to ef_{search}=300; within each such pair, blue/green use M=16 and red/brown use M=32. Across all three datasets, increasing ef_{search} shifts the trade-off upward and to the left: this is seen by comparing blue against green and red against brown. The interpretation is that a larger ef_{search} improves the native HNSW result, so a larger fraction of queries already satisfy the target recall before rectification, and less additional recovery work is needed. By contrast, the effect of increasing M at fixed ef_{search} is dependent on the level of compliance rate. At lower compliance levels, the larger-M configurations (red versus blue, and brown versus green) typically begin from a higher compliance rate at the same latency, suggesting that a denser graph yields a higher-recall baseline HNSW search. However, in the high-compliance regime, the advantage of larger M can diminish or reverse, as once rectification is invoked more frequently, the cost of each Dijkstra expansion grows with graph density as each expanded node exposes more neighbors. Consequently, for the same ef_{search}, the smaller-M configuration can eventually attain comparable compliance at lower latency. Overall, Figure 8 indicates that ef_{search} is the cleaner query-time knob, whereas the effect of M reflects a balance between better native search quality and more expensive recovery on denser graphs. A practical recommendation is therefore to first tune ef_{search} for latency-sensitive deployments, and to increase M primarily when the application benefits from a stronger baseline HNSW operating point and does not target only the extreme upper end of the compliance range. At the same compliance rate, LTT also has lower latency than CRC for 73.68% of the evaluated operating points.

The CTR framework naturally extends to filtered search on top of algorithms such as ACORN [42]. The overall trends are similar to those presented in the case of HNSW, with CTR over ACORN achieving the desired recall irrespective of the predicate selectivity, albeit with increased query time. Due to space constraints, we leave that result to the full version of this paper [6].

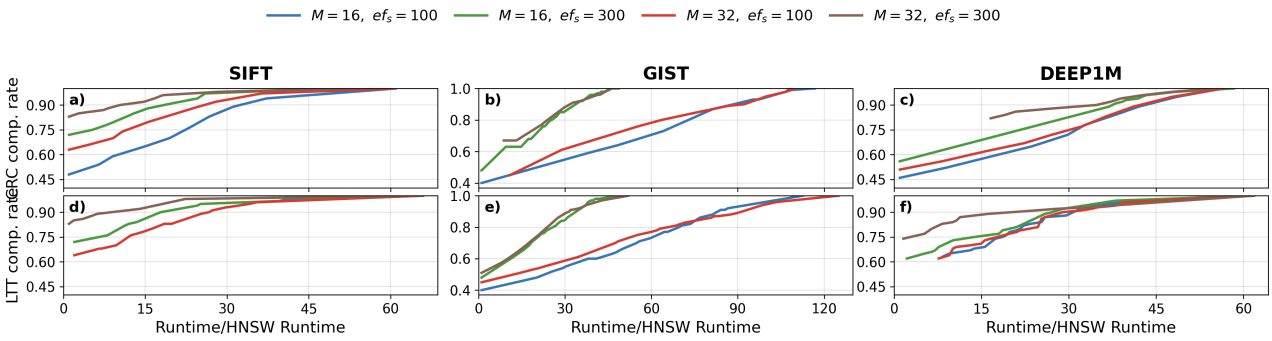

Figure 8: LTT HNSW-parameter ablation on three datasets. X-axis reports runtime as a multiple of HNSW runtime.

τ	DARTH		CTR	
	Default	Tuned	CRC	LTT
0.85	0.968	0.971	0.983	0.983
0.90	0.913	0.913	0.965	0.966
0.95	0.709	0.709	0.865	0.874

Table 4: Certifier comparison on SIFT1M: F1 Scores

7.5 DARTH vs. CRC/LTT

Recently, DARTH [13] was proposed solely as a predictor of recall. We therefore compare it directly against CRC and LTT in a certifier-only setting, i.e., without true neighbor rectification (as DARTH does not have this capability). Table 4 reports the F1 scores with respect to the binary event $Recall(q) \geq \tau$. For fairness, we use DARTH only as a recall predictor and let HNSW run to the same stopping point in all methods, rather than using DARTH's early-termination mechanism. The experiment is conducted on SIFT1M with M=32, ef_c=200, ef_s=100, k=100 and 1000 random queries, utilizing the implementation from the DARTH authors. Across all three target levels, CRC/LTT attain higher F1, with the gap becoming larger at stricter targets. Similar results were obtained across all data sets and are omitted for brevity. DARTH is a best-effort predictor and does not formally control the fraction of queries whose recall falls below the target. In contrast, CRC and LTT are calibrated for fixed deployment settings(dataset, index configuration, workload,and target recall τ) and explicitly control compliance-related errors under that setting. Consequently, for deciding whether an HNSW result should be accepted as meeting a prescribed recall target, CRC/LTT yield a more reliable acceptance rule.

8 RELATED WORK

A dominant family of graph indexing and ANN search algorithms stems from proximity-construction methods grounded in the geometry of relative neighborhood graphs (RNGs) and approximate Delaunay graphs [44, 48]. Key examples include navigable indexes such as Navigable Small-World graphs (NSW) and their hierarchical HNSW variant [38, 39], which constitute the state-of-the-art. Given its success, many variants of the vanilla HNSW have been proposed in recent literature [35, 42, 43, 58]. Related approaches further include navigating spread-out graphs (NSGs) [23] and the disk-based DiskANN index [28]. Efforts have also been made to accelerate kNN search by compressing the vector space using quantization [30] and inverted file (IVF) indexing [20, 29], and using locality-sensitive hashing [12, 18, 37]. A comprehensive survey on ANN search appears in [41]. For filtered search, the literature can be split into: (i) works on filter-aware index structures [10, 35, 52, 58]; (ii) standard ANN backbones with improved query-time execution [1, 42]; and (iii) multi-index approaches [34]. A survey of filtered search methods is given in [16]. Beyond improving the search algorithm itself, recent work has emphasized that query difficulty and workload composition strongly affect the observed behavior of ANN indexes. Wang et al. [49] introduced Steiner-hardness, a query hardness measure specifically designed for graph-based ANN indexes, while Ceccarello et al. [11] studied how to evaluate and generate query workloads of controlled difficulty for high-dimensional vector similarity search. Recent work further explores learned termination checks, where models map query-time features to per-query budgets [31, 53, 55]. Related declarative target-recall approaches terminate when the system *predicts* (heuristically) that the target recall has been met (e.g., DARTH [13]). Similar early-exit ideas also appear in non-graph pipelines [9, 14, 26, 40, 56]. In contrast to App2Exa [32], an exact KNN search method, we directly use the HNSW graph structure for rectified recovery rather than relying on a separate tree index. We also build on statistical models grounded in extreme value theory, including standard inferential toolkits for Weibull distributions [17, 22] and conformal prediction [3–5, 8]. Lastly, note that our formulations w.r.t. distance-call pruning take inspiration from recent literature on this topic [7, 19, 24, 31, 33, 45, 46, 51, 54].

9 FINAL REMARKS

This paper introduces a framework for generating correctness certificates in ANN search by wrapping HNSW in a "Certify-then-Rectify" pipeline. Using a Conformal Prediction formulation, we efficiently certify whether HNSW's top-k results miss closer neighbors. If certification fails, we trigger exact recovery to fetch true top-k neighbors. Our key novelty is reinterpreting HNSW's bottom layer as an empirical graph spanner to estimate a high-confidence stretch factor, bounding the exact recovery search radius. Recovery uses Stretch-Bounded Expansion from Query (SBE-Q) and Metric Bound Verification (MBV) to prune distance computations while preserving correctness. The framework supports filtered search without predicate-respecting paths.

REFERENCES

[1] Anas Ait Aomar, Karima Echihabi, Marco Arnaboldi, Ioannis Alagiannis, Damien Hilloulin, and Manal Cherkaoui. 2025. RWalks: Random Walks as Attribute Diffusers for Filtered Vector Search. *PACMMOD* 3, 3 (2025).

[2] Yousef Al-Jazzazi, Haya Diwan, Jinrui Gou, Cameron Musco, Christopher Musco, and Torsten Suel. 2025. Distance Adaptive Beam Search for Provably Accurate Graph-Based Nearest Neighbor Search. *arXiv preprint arXiv:2505.15636* (2025).

[3] Anastasios N Angelopoulos and Stephen Bates. 2021. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv preprint arXiv:2107.07511* (2021).

[4] Anastasios N. Angelopoulos, Stephen Bates, Emmanuel J. Candès, and Michael I. Jordan. 2024. Conformal Risk Control. *J. Amer. Statist. Assoc.* 119, 548 (2024), 2386–2408. https://doi.org/10.1080/01621459.2024.2302500

[5] Anastasios N. Angelopoulos, Stephen Bates, Michael I. Jordan, and Jitendra Malik. 2022. Learn then Test: Calibrating Predictive Algorithms to Achieve Risk Control. In *ICLR*. OpenReview.

[6] Anonymous Authors. 2026. *Recall Certification: Technical Report.* Technical Report. Anonymous Institution. https://anonymous.4open.science/r/recall_certification_anonymous-81BD/recall_certification_technical_report.pdf Anonymous technical report for double-blind review.

[7] Jees Augustine, Suraj Shetiya, Mohammadreza Esfandiari, Senjuti Basu Roy, and Gautam Das. 2021. A Generalized Approach for Reducing Expensive Distance Calls for A Broad Class of Proximity Problems. In *ACM SIGMOD*. 142–154.

[8] Stephen Bates, Anastasios Angelopoulos, Lihua Lei, Jitendra Malik, and Michael Jordan. 2021. Distribution-free, Risk-controlling Prediction Sets. *J. ACM* 68, 6, Article 43 (Sept. 2021), 34 pages. https://doi.org/10.1145/3478535

[9] Francesco Busolin, Claudio Lucchese, Franco Maria Nardini, Salvatore Orlando, Raffaele Perego, and Salvatore Trani. 2024. Early exit strategies for approximate k-NN search in dense retrieval. In *CIKM*. 3647–3652.

[10] Yuzheng Cai, Jiayang Shi, Yizhuo Chen, and Weiguo Zheng. 2024. Navigating Labels and Vectors: A Unified Approach to Filtered Approximate Nearest Neighbor Search. *PACMMOD* 2, 6 (2024).

[11] Matteo Ceccarello, Alexandra Levchenko, Ioana Ileana, and Themis Palpanas. 2025. Evaluating and Generating Query Workloads for High Dimensional Vector Similarity Search. In *ACM SIGKDD*.

[12] Moses S. Charikar. 2002. Similarity estimation techniques from rounding algorithms. In *STOC*. 380–388.

[13] Manos Chatzakis, Yannis Papakonstantinou, and Themis Palpanas. 2025. Darth: Declarative recall through early termination for approximate nearest neighbor search. *PACMMOD* 3, 4 (2025), 1–26.

[14] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. SPANN: Highly-Efficient Billion-Scale Approximate Nearest Neighborhood Search. In *Advances in Neural Information Processing Systems*, Vol. 34.

[15] Yilun Chen, Zhiding Yu, Yukang Chen, Shiyi Lan, Anima Anandkumar, Jiaya Jia, and Jose M. Alvarez. 2023. FocalFormer3D: Focusing on Hard Instance for 3D Object Detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*.

[16] Yannis Chronis, Helena Caminal, Yannis Papakonstantinou, Fatma Özcan, and Anastasia Ailamaki. 2025. Filtered Vector Search: State-of-the-Art and Research Opportunities. *PVLDB* 18, 12 (2025), 5488–5492.

[17] Stuart Coles, Joanna Bawa, Lesley Trenner, and Pat Dorazio. 2001. *An introduction to statistical modeling of extreme values.* Vol. 208. Springer.

[18] Mayur Datar, Nicole Immorlica, Piotr Indyk, and Vahab S. Mirrokni. 2004. Locality-sensitive hashing scheme based on p-stable distributions. In *SCG*. 253–262.

[19] Liwei Deng, Penghao Chen, Ximu Zeng, Tianfu Wang, Yan Zhao, and Kai Zheng. 2024. Efficient Data-Aware Distance Comparison Operations for High-Dimensional Approximate Nearest Neighbor Search. *PVLDB* 18, 3 (2024), 812–821.

[20] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The Faiss library. (2024). arXiv:2401.08281 [cs.LG]

[21] Hao Duan, Yitong Song, Bin Yao, and Anqi Liang. 2025. PGTuner: An Efficient Framework for Automatic and Transferable Configuration Tuning of Proximity Graphs. arXiv:2508.17886 [cs.DB] https://arxiv.org/abs/2508.17886

[22] R. A. Fisher and L. H. C. Tippett. 1928. Limiting forms of the frequency distribution of the largest or smallest member of a sample. *Mathematical Proceedings of the Cambridge Philosophical Society* 24, 2 (1928), 180–190.

[23] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast approximate nearest neighbor search with the navigating spreading-out graph. *PVLDB* 12, 5 (2019), 461–474.

[24] Jianyang Gao and Cheng Long. 2023. High-Dimensional Approximate Nearest Neighbor Search: with Reliable and Efficient Distance Comparison Operations. *PACMMOD* 1, 2 (2023).

[25] E. J. Gumbel. 1958. *Statistics of Extremes.* Columbia University Press, New York, Chichester, West Sussex. https://doi.org/10.7312/gumb92958

[26] Sonia Horchidan, Fabian Zeiher, Henrik Boström, and Paris Carbone. 2026. CoNANN: Conformal Approximate Nearest Neighbor Search. *PVLDB* 19, 1 (2026), 29–42.

[27] Physical Intelligence, Ali Amin, Raichelle Aniceto, Ashwin Balakrishna, Kevin Black, Ken Conley, Grace Connors, James Darpinian, Karan Dhabalia, Jared DiCarlo, Danny Driess, Michael Equi, Adnan Esmail, Yunhao Fang, Chelsea Finn, Catherine Glossop, Thomas Godden, Ivan Goryachev, Lachy Groom, Hunter Hancock, Karol Hausman, Gashon Hussein, Brian Ichter, Szymon Jakubczak, Rowan Jen, Tim Jones, Ben Katz, Liyiming Ke, Chandra Kuchi, Marinda Lamb, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Yao Lu, Vishnu Mano, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Charvi Sharma, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, Will Stoeckle, Alex Swerdlow, James Tanner, Marcel Torne, Quan Vuong, Anna Walling, Haohuan Wang, Blake Williams, Sukwon Yoo, Lili Yu, Ury Zhilinsky, and Zhiyuan Zhou. 2025. $\pi^*_{0.6}$: a VLA That Learns From Experience. arXiv:2511.14759 [cs.LG] https://arxiv.org/abs/2511.14759

[28] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *NeurIPS*, Vol. 32.

[29] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.

[30] Herve Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128.

[31] Conglong Li, Minjia Zhang, David G Andersen, and Yuxiong He. 2020. Improving approximate nearest neighbor search through learned adaptive early termination. In *ACM SIGMOD*. 2539–2554.

[32] Ke Li, Leong Hou U, and Shuo Shang. 2025. App2Exa: Accelerating Exact kNN Search via Dynamic Cache-Guided Approximation. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence (IJCAI-25)*. 3045–3053. https://doi.org/10.24963/ijcai.2025/339

[33] Zhenxin Li, Shuibing He, Jiahao Guo, Xuechen Zhang, Xian-He Sun, and Gang Chen. 2025. CRouting: Reducing Expensive Distance Calls in Graph-Based Approximate Nearest Neighbor Search. *arXiv preprint arXiv:2509.00365* (2025).

[34] Zhaoheng Li, Silu Huang, Wei Ding, Yongjoo Park, and Jianjun Chen. 2025. SIEVE: Effective Filtered Vector Search with Collection of Indexes. *PVLDB* 18, 11 (2025), 4723–4736.

[35] Anqi Liang, Pengcheng Zhang, Bin Yao, Zhongpu Chen, Yitong Song, and Guangxu Cheng. 2024. UNIFY: Unified Index for Range Filtered Approximate Nearest Neighbors Search. *PVLDB* 18, 4 (2024), 1118–1130.

[36] Zhenxi Lin, Ziheng Zhang, Xian Wu, and Yefeng Zheng. 2023. Improving Biomedical Entity Linking with Retrieval-Enhanced Learning. *arXiv preprint arXiv:2312.09806* (2023). https://doi.org/10.48550/arXiv.2312.09806

[37] Qin Lv, William Josephson, Zhe Wang, Moses Charikar, and Kai Li. 2007. Multi-probe LSH: efficient indexing for high-dimensional similarity search. In *VLDB*. 950–961.

[38] Yury Malkov, Alexander Ponomarenko, Andrey Logvinov, and Vladimir Krylov. 2014. Approximate nearest neighbor algorithm based on navigable small world graphs. *Information Systems* 45 (2014), 61–68.

[39] Yu A Malkov and Dmitry A Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions On Pattern Analysis And Machine Intelligence* 42, 4 (2018), 824–836.

[40] Jason Mohoney, Devesh Sarda, Mengze Tang, Shihabur Rahman Chowdhury, Anil Pacaci, Ihab F. Ilyas, Theodoros Rekatsinas, and Shivaram Venkataraman. 2025. Quake: Adaptive Indexing for Vector Search. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. USENIX Association.

[41] James Jie Pan, Jianguo Wang, and Guoliang Li. 2024. Survey of vector database management systems. *VLDB* 33, 5 (2024), 1591–1615.

[42] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data. *PACMMOD* 2, 3 (2024), 1–27.

[43] Zhencan Peng, Miao Qiao, Wenchao Zhou, Feifei Li, and Dong Deng. 2025. Dynamic Range-Filtering Approximate Nearest Neighbor Search. *PVLDB* 18, 10 (2025).

[44] Franco P Preparata and Michael I Shamos. 2012. *Computational Geometry: An Introduction.* Springer Science & Business Media.

[45] Yitong Song, Pengcheng Zhang, Chao Gao, Bin Yao, Kai Wang, Zongyuan Wu, and Lin Qu. 2025. TRIM: Accelerating High-Dimensional Vector Similarity Search with Enhanced Triangle-Inequality-Based Pruning. *PACMMOD* 3, 6 (2025), 1–26.

[46] Ziwen Song, Bin Wang, and Xiaochun Yang. 2025. Accelerating High-Dimensional ANN Search via Skipping Redundant Distance Computations. *PACMMOD* 3, 6 (2025), 1–29.

[47] Tommaso Teofili and Jimmy Lin. 2025. Patience In Proximity: A Simple Early Termination Strategy for HNSW Graph Traversal in Approximate k-Nearest Neighbor Search. In *ECIR*. 401–407.

[48] Godfried T. Toussaint. 1980. The relative neighbourhood graph of a finite planar set. *Pattern Recognition* 12, 4 (1980), 261–268.

[49] Zeyu Wang, Qitong Wang, Xiaoxing Cheng, Peng Wang, Themis Palpanas, and Wei Wang. 2024. Steiner-Hardness: A Query Hardness Measure for Graph-Based ANN Indexes. *PVLDB* 17, 13 (2024), 4668–4682.

[50] Maciej Wiatrak, Eirini Arvaniti, Angus Brayne, Jonas Vetterle, and Aaron Sim. 2023. Proxy-based Zero-Shot Entity Linking by Effective Candidate Retrieval. *arXiv preprint arXiv:2301.13318* (2023). https://doi.org/10.48550/arXiv.2301.13318

[51] Qian Xu, Juan Yang, Feng Zhang, Junda Pan, Kang Chen, Youren Shen, Amelie Chi Zhou, and Xiaoyong Du. 2025. Tribase: A Vector Data Query Engine for Reliable and Lossless Pruning Compression using Triangle Inequalities. *PACMMOD* 3, 1 (2025), 1–28.

[52] Yuexuan Xu, Jianyang Gao, Yutong Gou, Cheng Long, and Christian S. Jensen. 2024. iRangeGraph: Improvising Range-dedicated Graphs for Range-filtering Nearest Neighbor Search. *PACMMOD* 2, 6 (2024).

[53] Kaixiang Yang, Hongya Wang, Bo Xu, Wei Wang, Yingyuan Xiao, Ming Du, and Junfeng Zhou. 2021. Tao: A learning framework for adaptive nearest neighbor search using static features only. *arXiv preprint arXiv:2110.00696* (2021).

[54] Mingyu Yang, Wentao Li, Jiabao Jin, Xiaoyao Zhong, Xiangyu Wang, Zhitao Shen, Wei Jia, and Wei Wang. 2025. Effective and general distance computation for approximate nearest neighbor search. In *IEEE ICDE*. 1098–1110.

[55] Chao Zhang and Renée J. Miller. 2025. Distribution-Aware Exploration for Adaptive HNSW Search. arXiv:2512.06636 [cs.DB] https://arxiv.org/abs/2512.06636

[56] Zili Zhang, Chao Jin, Linpeng Tang, Xuanzhe Liu, and Xin Jin. 2023. Fast, Approximate Vector Queries on Very Large Unstructured Datasets. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, 995–1011.

[57] Xiaoyao Zhong, Haotian Li, Jiabao Jin, Mingyu Yang, Deming Chu, Xiangyu Wang, Zhitao Shen, Wei Jia, George Gu, Yi Xie, Xuemin Lin, Heng Tao Shen, Jingkuan Song, and Peng Cheng. 2025. VSAG: An Optimized Search Framework for Graph-Based Approximate Nearest Neighbor Search. *PVLDB* 18, 12 (2025), 5017–5030.

[58] Chaoji Zuo, Miao Qiao, Wenchao Zhou, Feifei Li, and Dong Deng. 2024. SeRF: Segment Graph for Range-Filtering Approximate Nearest Neighbor Search. *PACMMOD* 2, 1 (2024).